

Exqutor 기반 캡스톤 — 연구 기획안

작성일: 2026-03-27 (최종 수정: 2026-03-28)

목적: 팀 토의용. Exqutor 논문의 한계/미검증 영역에서 캡스톤 수준으로 진행 가능한 연구 방향 후보를 최대한 넓게 수집.

배경: 교수님 연구실에서 Exqutor 데모 페이퍼 마무리 단계. 우리 팀은 별도로 주제를 가져가서 사이드 토이 프로젝트로 진행해야 함.

3/28 업데이트: 팀 토의(강재현 아이디어) 반영하여 J 카테고리(Cascaded Decomposition) 추가 및 추천 순위 갱신.

I. 논문이 명시한 한계 (우리가 파고들 여지)

Exqutor Section 6.8에서 저자들이 직접 인정한 한계 3가지:

1. **고차원 공간에서 샘플링 오버헤드 증가** — 차원이 높을수록 거리 계산 비용이 커져 샘플링 효율이 떨어짐
2. **기존 비용 모델의 부정확성** — 정확한 카디널리티를 줘도 DB의 비용 모델이 ANN 인덱스 특성(로그 복잡도 등)을 반영 못해서 차선의 실행 계획 선택 가능
3. **벡터 인덱스 맞춤 비용 모델 부재** — 향후 과제로만 언급하고 해결하지 않음

II. 연구 방향 후보 목록

A. 비용 모델 계열 — "카디널리티가 정확해도 계획이 틀리는 문제"

A-1. 비용 모델 한계 실증 분석

- **질문:** Exqutor가 정확한 카디널리티를 줬는데도 옵티마이저가 최적이 아닌 계획을 선택하는 빈도는?
- **방법:** pg_hint_plan으로 모든 가능한 조인 방식(Nested Loop / Hash / Merge) × 스캔 방식(Seq / Index / Bitmap) 조합을 강제 지정 → 실제 실행 시간 측정 → oracle plan(진짜 최적) 확인 → Exqutor 선택 계획과 비교
- **기대 산출물:** "비용 모델이 틀리는 패턴 분류표", 어떤 조건에서 비용 모델이 가장 많이 틀리는지
- **난이도:** 중 (pg_hint_plan 사용법 학습 필요, 코드 수정 없음)
- **장점:** 논문의 명시적 한계를 직접 검증. 교수님 입장에서 후속 연구에 실질적 데이터 제공
- **단점:** pg_hint_plan 조합이 많아서 자동화 스크립트 필요

A-2. 벡터 인덱스 비용 모델 보정 규칙 제안

- **질문:** HNSW Index Scan의 실제 비용을 PostgreSQL 비용 모델에 어떻게 반영하면 계획 선택이 개선되는가?
- **방법:** A-1 결과에서 비용 모델이 틀리는 패턴을 분석 → PostgreSQL의 비용 상수(seq_page_cost, random_page_cost, cpu_tuple_cost 등)를 벡터 연산에 맞게 조정하는 규칙 제안 → 조정 전/후 계획 품질 비교
- **기대 산출물:** "벡터 워크로드용 PostgreSQL 비용 상수 튜닝 가이드"
- **난이도:** 중~상 (PostgreSQL 비용 모델 내부 이해 필요)
- **장점:** 실용적 가치 높음. 실제 pgvector 사용자에게 바로 적용 가능
- **단점:** PostgreSQL 내부 지식 학습 곡선

A-3. 조인 순서 최적화 — 벡터 필터 위치의 영향

- **질문:** 다중 조인 VAQ에서 벡터 필터를 조인 트리의 어느 위치에 배치하는 것이 최적인가?
- **방법:** 4~5개 테이블 조인 VAQ에서 벡터 필터를 (1) 가장 먼저 (2) 중간 (3) 마지막에 적용하도록 pg_hint_plan 으로 강제 → 실행 시간 비교
- **기대 산출물:** "벡터 필터 최적 위치 결정 규칙"
- **난이도:** 중
- **장점:** 재현이 아이디어("많이 retrieval → 적게 retrieval")와 직접 연결
- **단점:** 조인 수가 적으면 차이가 안 날 수 있음

B. 선택도 분석 계열 — "어떤 조건에서 Exqutor가 효과적인가"

B-1. 선택도 sweep — Exqutor 효과의 경계 조건 매핑

- **질문:** 벡터 선택도(결과 비율)를 연속적으로 변화시키면 Exqutor 효과가 어떻게 달라지는가?
- **방법:** 거리 임계값 D를 조절해서 선택도를 0.01% → 0.1% → 1% → 5% → 10% → 33% → 50% → 90%로 sweep. 각 지점에서 (1) Q-error (2) 선택된 실행 계획 (3) 실행 시간 측정
- **기대 산출물:** 선택도 vs 실행시간 곡선, "pgvector 33.3% 고정 추정치 우연히 맞는 교차점" 시각화, Exqutor 효과 영역 매핑
- **난이도:** 하~중 (SQL의 D값만 변경, 자동화 쉬움)
- **장점:** 논문에 없는 체계적 분석. baseline으로 다른 실험의 기반이 됨. 자문위원 피드백(데이터셋/대조군/지표)을 한 번에 충족
- **단점:** 분석 자체가 단순할 수 있음. 다른 방향과 결합해야 깊이가 생김

B-2. 데이터 분포별 영향 분석

- **질문:** 벡터 데이터의 분포 특성(균일/클러스터/편향)에 따라 카디널리티 추정 오차 패턴이 어떻게 달라지는가?
- **방법:**
 - 균일 분포 벡터 (랜덤 생성)
 - 강한 클러스터 벡터 (K-means로 10~100개 군집 생성)
 - 편향 분포 벡터 (일부 영역에 집중)
 - 실제 임베딩 (OpenAI, SIFT 등)
 - 각 분포에서 동일한 선택도 sweep 실행
- **기대 산출물:** "데이터 분포 × 선택도" 2차원 히트맵, 클러스터링 정도와 Q-error의 상관관계
- **난이도:** 중 (합성 데이터 생성 필요)
- **장점:** Exquator의 적응적 샘플링이 정말로 다양한 분포에 적응하는지 검증
- **단점:** 합성 데이터의 현실성 한계

B-3. 차원별 영향 분석

- **질문:** 벡터 차원(96 → 128 → 256 → 768 → 1536)이 증가하면 카디널리티 추정과 쿼리 성능에 어떤 영향?
- **방법:** 동일한 쿼리를 다른 차원의 벡터 데이터셋으로 실행. 차원별 (1) ECQO 오버헤드 (2) 샘플링 정확도 (3) 인덱스 빌드/검색 시간 비교
- **기대 산출물:** 차원 vs 성능 곡선, "차원의 저주가 카디널리티 추정에 미치는 영향" 분석
- **난이도:** 중 (다양한 차원의 데이터셋 준비 필요)
- **장점:** 논문의 한계 #1(고차원 오버헤드)을 직접 검증
- **단점:** 고차원 데이터셋 확보가 번거로울 수 있음

C. 필터링 전략 계열 — "벡터 + 관계형 복합 조건"

C-1. 복합 필터 조합의 카디널리티 오차 증폭 실험

- **질문:** 벡터 필터에 관계형 필터를 겹칠수록 카디널리티 추정 오차가 어떻게 증폭되는가?
- **방법:** 필터 단계적 추가: (1) 벡터만 (2) +날짜 범위 (3) +카테고리 label (4) +가격 범위 (5) +텍스트 LIKE. 각 단계에서 Q-error와 실행 시간 측정
- **기대 산출물:** "필터 수 vs Q-error" 증가 곡선, 오차 증폭 패턴 분류
- **난이도:** 중 (쿼리 설계에 신경 필요)
- **장점:** 실무 RAG에서 복합 필터가 기본인데 논문은 단독/태그 1개만 테스트. 현실 갭을 직접 측정
- **단점:** 필터 조합이 많아서 실험 행렬이 커질 수 있음

C-2. Pre-filter vs Post-filter vs Hybrid 전략 비교

- **질문:** 관계형 필터를 벡터 검색 전에 적용(pre-filter), 후에 적용(post-filter), 혼합 적용(hybrid) 중 어떤 것이 최적인가?
- **방법:**
 - Pre-filter: 관계형 조건으로 먼저 걸러낸 후 벡터 검색
 - Post-filter: 벡터 검색 결과에서 관계형 조건으로 필터
 - Hybrid: 일부 필터는 선행, 일부는 후행
- 선택도 조합별로 어떤 전략이 우세한지 매핑
- **기대 산출물:** "관계형 선택도 × 벡터 선택도" 매트릭스에서의 최적 전략 맵
- **난이도:** 중 (pg_hint_plan 또는 서브쿼리 구조로 순서 제어)
- **장점:** 재현 아이디어의 일반화 버전. ACORN, SeRF 등 필터링 논문과 연결 가능
- **단점:** pgvector에서 필터 순서를 명시적으로 제어하기 어려울 수 있음 (옵티마이저가 개입)

C-3. 상관관계 있는 필터에서의 카디널리티 추정

- **질문:** 벡터 유사도와 관계형 속성 사이에 상관관계가 있을 때 (예: 비슷한 이미지는 비슷한 가격대) 카디널리티 추정이 더 틀리는가?
- **방법:** 의도적으로 상관관계를 가진 데이터 생성 (벡터 거리와 가격이 비례하도록) → 독립적 데이터와 비교 실험
- **기대 산출물:** "속성 간 상관관계가 Q-error에 미치는 영향" 분석
- **난이도:** 중~상 (상관 데이터 생성 + 분석)
- **장점:** 실제 데이터에서 벡터와 속성이 독립이 아닌 경우가 많음. 현실적 시나리오
- **단점:** 상관관계 정도를 정밀하게 제어하기 까다로움

D. 인덱스/시스템 계열 — "HNSW 말고 다른 조건에서는?"

D-1. IVFFlat vs HNSW — 인덱스 유형별 ECQO 효과 비교

- **질문:** pgvector의 IVFFlat 인덱스에서도 ECQO 접근법이 유효한가?
- **방법:** 동일한 데이터/쿼리에 대해 HNSW vs IVFFlat 인덱스를 각각 빌드 → EXPLAIN ANALYZE 비교 → 카디널리티 추정 행동 차이 분석
- **기대 산출물:** 인덱스 유형별 카디널리티 추정 정확도 + 성능 비교표
- **난이도:** 하~중 (인덱스 타입 변경만 하면 됨)
- **장점:** Exquator가 HNSW만 테스트한 것의 빈자리를 직접 채움

- **단점:** IVFFlat에서 range query 지원이 제한적일 수 있음

D-2. HNSW 파라미터 민감도 분석

- **질문:** HNSW의 M, ef_construction, ef_search 파라미터가 ECQO의 카디널리티 정확도에 얼마나 영향을 주는가?
- **방법:** 파라미터 조합을 변경하면서 (1) 인덱스 빌드 시간 (2) ECQO 카디널리티 정확도 (3) 쿼리 실행 시간 측정
- **기대 산출물:** "HNSW 파라미터 × ECQO 효과" 3D 플롯, 최적 파라미터 추천 가이드
- **난이도:** 하~중 (파라미터 변경 + 반복 실험)
- **장점:** 실무 튜닝 가이드로서의 가치. 논문은 M=16, ef_construction=200, ef_search=400 고정만 테스트
- **단점:** 파라미터 공간이 넓어서 전수 탐색은 시간 소모적

D-3. pgvector vs DuckDB 직접 비교 (동일 조건)

- **질문:** 동일한 데이터셋과 동일한 쿼리에서 pgvector와 DuckDB의 카디널리티 추정 행동과 실행 계획이 어떻게 다른가?
- **방법:** SIFT1M 같은 소규모 데이터셋으로 양쪽에서 동일 쿼리 실행 → 실행 계획 구조 비교 → 각 시스템의 카디널리티 추정 로직 차이 분석
- **기대 산출물:** 시스템 간 비교표, 각 시스템의 강점/약점 정리
- **난이도:** 중 (두 시스템 모두 셋업 필요)
- **장점:** 논문에서 각 시스템을 별도로 테스트했지만 "동일 조건 직접 비교"는 하지 않음
- **단점:** DuckDB 벡터 확장(duckdb-vss) 셋업이 추가적

E. Top-k 게열 — "논문이 의도적으로 안 다룬 영역"

E-1. Top-k 쿼리의 카디널리티 추정 문제

- **질문:** range query가 아닌 top-k(`ORDER BY distance LIMIT k`)에서도 카디널리티 오추정 문제가 존재하는가?
- **방법:** 동일한 VAQ를 range와 top-k 두 버전으로 작성 → pgvector에서 각각 EXPLAIN ANALYZE → 카디널리티 추정 행동 비교
- **기대 산출물:** "top-k에서는 k가 고정이므로 카디널리티 추정 문제가 다른 형태로 나타남" 분석
- **난이도:** 하 (쿼리 변환만)
- **장점:** 실무에서 top-k가 압도적으로 많이 쓰임. 논문이 range만 다룬 이유 검증
- **단점:** top-k에서는 LIMIT이 카디널리티를 결정하므로 문제의 성격이 다를 수 있음

E-2. Top-k에서의 조인 성능 비교

- **질문:** top-k 벡터 검색이 포함된 다중 조인 쿼리에서 pgvector가 효율적인 계획을 세우는가?
- **방법:** top-k VAQ에서 k값을 1, 10, 100, 1000, 10000으로 변경하면서 실행 계획 변화 관찰
- **기대 산출물:** k값에 따른 실행 계획 전환점(crossover point) 분석
- **난이도:** 하~중
- **장점:** top-k에서도 옵티마이저 문제가 있는지 확인. 있다면 Exqutor 확장 방향 제시 가능
- **단점:** top-k + 조인 쿼리 구조가 range + 조인과 다르게 동작할 수 있음

F. 실제 데이터/워크로드 계열 — "합성 vs 실제"

F-1. 실제 RAG 임베딩으로 검증

- **질문:** Exqutor는 DEEP/SIFT 등 연구용 벡터로만 테스트. 실제 OpenAI/Sentence-Transformers 임베딩에서도 동일한 효과?
- **방법:** 실제 텍스트 데이터(Wikipedia 일부, 뉴스 기사 등)를 sentence-transformers로 임베딩 → pgvector에 로딩 → 동일한 선택도 sweep 실행
- **기대 산출물:** "연구용 벡터 vs 실제 임베딩"의 분포 차이 시각화, 카디널리티 추정 정확도 비교
- **난이도:** 중 (임베딩 생성 파이프라인 구축 필요)
- **장점:** 실무 관련성 극대화. 실제 RAG 사용자에게 직접적인 참고 자료
- **단점:** 임베딩 생성에 GPU 또는 API 비용 필요

F-2. 거리 메트릭별 영향 분석

- **질문:** L2(유클리드) 외에 코사인 유사도, 내적(inner product)에서 카디널리티 추정 행동이 달라지는가?
- **방법:** 동일 데이터에 대해 L2, 코사인, 내적 각각으로 인덱스 빌드 → 동일 쿼리의 EXPLAIN 비교
- **기대 산출물:** 메트릭별 카디널리티 추정 오차 비교표
- **난이도:** 하~중 (거리 연산자만 변경)
- **장점:** pgvector가 세 메트릭 모두 지원. 논문은 L2만 사용
- **단점:** 코사인과 L2는 정규화된 벡터에서 동치이므로 차이가 크지 않을 수 있음

F-3. 데이터 규모별 효과 분석 (소규모 집중)

- **질문:** Exqutor는 SF100(대규모)에서 테스트. 소규모(1만~100만건)에서도 카디널리티 오추정이 문제가 되는가?
- **방법:** 10K, 100K, 1M, 10M 규모로 단계적 테스트 → 규모별 Exqutor 효과 곡선

- **기대 산출물:** "어느 규모부터 카디널리티 추정이 중요해지는가" 임계점 분석
- **난이도:** 하 (데이터 샘플링으로 규모 조절)
- **장점:** 우리 팀 하드웨어 제약에 맞춤. 소규모에서 시작해서 점진적 확장 가능
- **단점:** 소규모에서는 Full Table Scan이 오히려 빠를 수 있어 차이가 안 날 수 있음

G. 적응적 샘플링 계열 — "인덱스 없는 환경"

G-1. 샘플링 하이퍼파라미터 민감도 분석

- **질문:** 적응적 샘플링의 모멘텀(0.9), 학습률 감소, 업데이트 주기(50쿼리)가 최적인가?
- **방법:** 각 하이퍼파라미터를 변경하면서 Q-error 수렴 속도와 최종 정확도 비교
- **기대 산출물:** 하이퍼파라미터 민감도 분석표, 최적 설정 추천
- **난이도:** 중~상 (Exqutor 코드 수정 필요)
- **장점:** 논문이 고정값만 사용한 것의 타당성 검증
- **단점:** Exqutor 소스코드 이해 + 수정이 필요

G-2. 대안적 샘플링 전략 비교

- **질문:** 랜덤 샘플링 외에 층화 샘플링(stratified), 중요도 샘플링(importance) 등을 적용하면 정확도가 개선되는가?
- **방법:** 다양한 샘플링 전략을 구현하여 동일 조건에서 Q-error 비교
- **기대 산출물:** 샘플링 전략별 정확도/오버헤드 비교표
- **난이도:** 상 (샘플링 전략 구현 + 실험)
- **장점:** 학술적 기여 가능성
- **단점:** 구현 난이도 높음. 한 학기 내 완성 불확실

G-3. 히스토그램 기반 카디널리티 추정

- **질문:** 벡터 거리 분포에 대한 히스토그램을 미리 구축해두면, 샘플링보다 빠르고 정확한 추정이 가능한가?
- **방법:** 쿼리 벡터 없이도 "데이터 내 벡터 쌍 거리 분포"를 히스토그램으로 저장 → 쿼리 시점에 히스토그램 참조로 카디널리티 추정 → 샘플링/ECQO와 정확도 비교
- **기대 산출물:** "히스토그램 vs 샘플링 vs ECQO" 3가지 전략 비교
- **난이도:** 중~상 (히스토그램 구축 로직 구현 필요)
- **장점:** Exqutor가 제안하지 않은 완전히 새로운 접근법
- **단점:** 쿼리 벡터 의존적이라 히스토그램의 일반성에 한계

H. 동적/운영 환경 계열

H-1. 데이터 업데이트 시 카디널리티 유효성

- **질문:** INSERT/UPDATE가 일어나는 환경에서 ECQO로 얻은 카디널리티가 얼마나 빨리 무효화되는가?
- **방법:** 벡터 데이터를 점진적으로 추가하면서 ECQO 카디널리티의 유효 기간 측정
- **기대 산출물:** "데이터 변경률 vs 카디널리티 유효성" 곡선
- **난이도:** 중
- **장점:** 실무에서 데이터는 계속 변함. 정적 데이터만 가정한 논문의 맹점
- **단점:** ECQO는 쿼리 시점에 매번 실행하므로 이 문제가 크지 않을 수 있음 (캐시 무효화가 핵심)

H-2. 동시 쿼리(Concurrent) 환경 성능

- **질문:** 다수 사용자가 동시에 VAQ를 실행하면 ECQO 오버헤드가 누적되는가?
- **방법:** pgbench 또는 Python 멀티스레드로 동시 쿼리 부하 생성 → 동시성 수준별 응답 시간 측정
- **기대 산출물:** "동시 사용자 수 vs 응답 시간" 곡선
- **난이도:** 중~상 (부하 테스트 환경 구축)
- **장점:** 논문이 전혀 다루지 않은 영역. 프로덕션 적용 시 필수 고려사항
- **단점:** 하드웨어 성능에 크게 의존. 우리 장비로 유의미한 결과가 나올지 불확실

I. 시각화/도구 계열 — "분석 결과를 보여주는 방법"

I-1. 쿼리 계획 비교 시각화 도구

- **질문:** EXPLAIN ANALYZE 결과를 체계적으로 비교/시각화하는 도구가 있으면 모든 실험의 분석이 쉬워지지 않나?
- **방법:** Python + Jupyter로 EXPLAIN JSON 출력을 파싱 → 트리 시각화 + 비용/실행시간 비교 차트 자동 생성
- **기대 산출물:** 재사용 가능한 분석 도구 + 모든 실험 결과의 시각화
- **난이도:** 하~중 (Python 스크립팅)
- **장점:** 다른 모든 연구 방향의 분석 효율을 높임. 보고서/발표에 직접 활용
- **단점:** 도구 자체는 연구 기여라 보기 어려움. 보조 수단

I-2. 벤치마크 자동화 프레임워크

- **질문:** 선택도 sweep, 파라미터 변경, 시스템 비교 등 반복 실험을 자동화할 수 있는가?

- **방법:** Python 스크립트로 (1) 데이터 로딩 (2) 인덱스 빌드 (3) 쿼리 실행 (4) EXPLAIN 파싱 (5) 결과 CSV 저장 파이프라인 구축
- **기대 산출물:** 재사용 가능한 벤치마크 프레임워크
- **난이도:** 중
- **장점:** 한 번 구축하면 모든 후속 실험이 빨라짐
- **단점:** 프레임워크 구축에 시간 투자 필요

J. Cascaded Decomposition 계열 — "벡터 조건 자체를 구조적으로 분해" (3/28 추가)

배경: 강재현 아이디어에서 출발. 기존 방향(A~I)이 "Exqutor가 추정한 카디널리티를 어떻게 활용하는가"에 집중한다면, J 계열은 "카디널리티를 추정해야 하는 부담 자체를 줄이는 구조적 접근"이라는 새로운 축을 제시한다. Exqutor(ECQO)와 직교하는 최적화 방향이므로, 보완 가능성이 있다.

J-1. VAQ 플래닝 병목 프로파일링

- **질문:** VAQ의 전체 지연 중 Planning Time이 차지하는 비중은 얼마인가? 벡터 조건의 선택도와 필터 복잡도에 따라 어떻게 변하는가?
- **방법:** EXPLAIN (ANALYZE, TIMING)으로 Planning/Execution 분리 측정. 선택도 0.01%~90%, 필터 0~3개, 조인 유무에 따른 Planning Time 비율 매핑
- **기대 산출물:** "Planning vs Execution 비율" 히트맵, "플래닝이 병목인 영역" 식별
- **난이도:** 하~중 (EXPLAIN ANALYZE만으로 가능)
- **장점:** 논문에 없는 분석. J-2의 전제 조건 검증. 단독으로도 가치 있는 프로파일링 데이터
- **단점:** 만약 Planning Time이 항상 미미하면 J-2의 전제가 무너지짐

J-2. Cascaded Vector Similarity Decomposition ★

- **질문:** 벡터 유사도 조건을 느슨한 임계값(많이 retrieval) → 엄격한 임계값(적게 retrieval)의 두 단계로 분해하면, 플래닝 시간이 감소하고 총 비용이 줄어드는가?
- **방법:**
 - 단일 쿼리(`cosine > 0.8`)를 CTE 기반 2단계로 분해(`Stage1: cosine > 0.3` → `Stage2: cosine > 0.8`)
 - 분해 임계값 간격을 변화시키며 Planning Time + Execution Time = Total Time 최적점 탐색
 - ECQO only / Cascade only / ECQO+Cascade 결합의 3가지 전략 비교
 - 중간 결과 캐싱 전략(CTE / Materialized CTE / Temp Table+Index) 비교
- **기대 산출물:** 분해 임계값 최적점, ECQO와의 보완성/대체성 판별, 캐싱 전략 가이드
- **난이도:** 중 (CTE 기반 쿼리 설계 + 다양한 조합 실험)

- **장점:** 팀 자체 아이디어. Exqutor와 직교하는 새로운 최적화 축. 논문에 전혀 없는 접근
- **단점:** J-1에서 플래닝이 병목이 아니면 효용성 제한. PostgreSQL CTE 인라인 최적화(optimization fence 제거)로 분해 효과가 사라질 수 있음 → MATERIALIZED 키워드로 대응 가능

J-3. Cascade + 다중 필터 상호작용

- **질문:** J-2의 Cascade에 관계형 필터가 추가되면 효과가 강화되는가, 약화되는가?
- **방법:** J-2 실험에 관계형 필터를 단계적 추가(0~3개). 필터가 많아질수록 플래닝 공간이 커지므로 Cascade 효과가 커지는지 검증
- **기대 산출물:** "필터 수 × Cascade 효과" 곡선
- **난이도:** 중 (J-2의 자연 확장)
- **장점:** 실무에서 다중 필터가 기본인 점을 반영
- **단점:** J-2에 의존적

III. 추천 순위 (현실성 + 임팩트)

순 위	방향	핵심 이유
1	J-1 (플래닝 병목 프로파일링)	J-2의 전제 검증 + 단독으로도 논문에 없는 분석. B-1 선택도 sweep을 포함
2	J-2 (Cascaded Decomposition) ★	팀 자체 아이디어. Exqutor와 직교하는 새 축. 3/28 팀 토의에서 선정된 핵심 방향
3	J-3 (Cascade + 다중 필터)	J-2의 자연 확장. 실무 관련성 높음
4	B-1 (선택도 sweep)	J-1에 포함 가능. 모든 실험의 baseline
5	A-1 (비용 모델 실증)	논문 명시 한계 직접 검증. pg_hint_plan으로 코드 수정 없음
6	C-1 (복합 필터 오차 증폭)	실무 관련성 높음. J-3과 시너지
7	E-1 (Top-k 카디널리티)	난이도 최하. Cascade의 top-k 확장 가능성 검증
8	D-2 (HNSW 파라미터 민감도)	실무 튜닝 가이드 가치
9	C-2 (Pre/Post-filter 비교)	필터링 논문들과 연결 가능
10	F-2 (거리 메트릭별)	쉽지만 차이가 크지 않을 가능성

순 위	방향	핵심 이유
11	B-3 (차원별 영향)	논문 한계 #1 검증
12	F-1 (실제 임베딩 검증)	실무 관련성 높지만 파이프라인 필요
13	D-1 (IVFFlat 비교)	IVFFlat range query 지원 제한
14	A-3 (벡터 필터 위치)	J-2와 일부 겹침
15	G-3 (히스토그램 기반)	구현 난이도 높음
16	I-1, I-2 (도구)	보조 수단
17+	나머지	난이도/의존성/하드웨어 제약

IV. 추천 조합 패키지

패키지 1: "Cascaded Decomposition" ★ (3/28 팀 토의 선정 방향)

J-1 → J-2 → J-3 + 확장(E-1 또는 D-2) - 병목 프로파일링 → Cascade 핵심 실험 → 다중 필터 확장 → top-k 또는 HNSW 파라미터 - 장점: 팀 자체 아이디어. Exqutor와 직교하는 새로운 축. 보완/대체 관계 판별이 스토리라인 - 단점: J-1에서 플래닝이 병목이 아닌 것으로 나오면 방향 전환 필요 → 패키지 2로 전환

패키지 2: "비용 모델 검증" (폴백 옵션)

B-1 → A-1 → C-1 - 선택도 sweep으로 baseline 확립 → 비용 모델 한계 실증 → 복합 필터 확장 - 장점: 논문의 핵심 한계를 체계적으로 파고듦. 스토리라인이 명확. J-1 결과와 무관하게 유효 - 단점: pg_hint_plan 학습 필요. A-1 이 실험량이 많음

패키지 3: "선택도 중심 분석" (안전한 선택)

B-1 + E-1 + F-2 - 선택도 sweep (메인) + top-k 비교 (부가) + 거리 메트릭 비교 (부가) - 장점: 전부 SQL 수준 실험. 코드 수정 없음. 4월 안에 끝낼 수 있음 - 단점: 깊이가 부족할 수 있음

패키지 4: "HNSW 파라미터 튜닝 가이드" (실용 중심)

B-1 → D-2 → D-1 → F-2 - 선택도 sweep → HNSW 파라미터 → IVFFlat 비교 → 메트릭 비교 - 장점: pgvector
사용자를 위한 실용적 가이드. 실험 설계가 단순반복 - 단점: 학술적 깊이 부족

V. 토의 포인트 (팀원들과 논의할 질문)

1. 우리 하드웨어는? 연구실 서버를 쓸 수 있는지에 따라 데이터 규모 결정
2. Exqutor 코드를 직접 수정하는 건 범위 내인가? → G계열 가능 여부 결정
3. 교수님이 기대하시는 산출물 형태는? 벤치마크 보고서 / 개선 제안 / 코드 기여 중 어느 쪽?
4. DuckDB도 같이 볼 건가? → D-3 가능 여부
5. 4월 세미나에서 발표할 내용은? → 4월 초까지 끝낼 수 있는 범위
6. 2명×2조 분담이면 환경구축조 + 분석조? 아니면 주제별 분담?